

Vendor Module – Database Structure v3

With Inventory Management & Future-Proof Scalability

School Uniform & Supplies Marketplace – Revised Design (June 2026)

This revision adds proper inventory management and scalability for future growth, on top of the earlier generic Product + Category + School Mapping structure. Two main additions: (1) a Product Variant table so stock is tracked per size/color/SKU instead of one stock_qty per product, and a Stock Ledger table for full audit history; (2) Warehouse/Location support and multi-vendor scaling fields so the platform can grow to many vendors, multiple warehouses, and new product types without schema changes.

1. Product Variant Table (NEW – Inventory at SKU Level)

Previously, stock_qty and sizes_available lived directly on the Product table as a single number/JSON array. For real inventory management, each size/color combination needs its own stock count and SKU. This table breaks a Product into individual sellable variants (e.g. "School Shirt - Size 32 - White").

Field Name	Data Type	Description	Validation / Notes
variant_id	UUID	Unique variant identifier	Auto-generated, Primary Key
product_id	FK → product_id	Parent product this variant belongs to	Required (Foreign Key)
sku	VARCHAR(50)	Stock Keeping Unit code, e.g. SHIRT-BAGGEE-001	Required, Unique
size_value	VARCHAR(20)	e.g. S, M, 32, UK-7, or 'NA' for non-sized items (pens, books)	Required
color / attribute	VARCHAR(50)	Optional secondary attribute (color, pack size, etc)	Optional, generic for future attributes
barcode	VARCHAR(50)	EAN/UPC barcode for scanning	Optional, future POS integration
price_override	DECIMAL(10,2)	Variant-specific price if different from base product, falls back to product.price	Optional, nullable
stock_qty	INT	Current available stock for THIS variant	Non-negative, moved here from Product table
reserved_qty	INT	Stock reserved by unconfirmed/pending orders	NEW, prevents overselling
low_stock_threshold	INT	Trigger value for low-stock alert notifications	NEW, default e.g. 5
is_active	BOOLEAN	Variant available for sale	Default: true
created_at / updated_at	DATETIME	Record timestamps	Auto-set / auto-updated

Product table change: remove stock_qty and sizes_available fields — these are now derived from the Product Variant table (one product has many variants).

2. Stock Ledger Table (NEW – Inventory Movement History)

Every stock change — vendor adding stock, an order reserving/deducting stock, a return adding stock back, or admin manual adjustment — is recorded as a row here. This gives a full audit trail and lets stock_qty on the Variant table always be recalculated/verified.

Field Name	Data Type	Description	Validation / Notes
ledger_id	UUID	Unique ledger entry	Auto-generated, Primary Key
variant_id	FK → variant_id	Which product variant this entry affects	Required (Foreign Key)
warehouse_id	FK → warehouse_id	Which location/warehouse stock moved at	Required (Foreign Key) – see Section 3
movement_type	ENUM	stock_in / stock_out / order_reserved / order_fulfilled / return_in / adjustment / damaged	Required
quantity_change	INT	Positive or negative change applied	Required, can be +/-
reference_type	ENUM	order / purchase / return / manual / settlement	Required – what triggered this
reference_id	UUID	ID of the order/purchase/return record that triggered this	Optional, links to Order etc.
balance_after	INT	Resulting stock_qty after this movement	Calculated, for audit/debug
performed_by	FK → user (vendor/admin)	Who triggered this movement	Optional, for accountability
notes	TEXT	Free text remark (e.g. 'damaged in transit')	Optional
created_at	DATETIME	Timestamp of movement	Auto-set

3. Warehouse / Location Table (NEW – Multi-Location Scaling)

Even if a vendor currently has a single store, adding a Warehouse table now avoids a painful migration later when vendors expand to multiple cities/godowns, or when the platform introduces a central fulfilment center.

Field Name	Data Type	Description	Validation / Notes
warehouse_id	UUID	Unique location identifier	Auto-generated, Primary Key
vendor_id	FK → vendor_id	Owning vendor (null/platform-owned for central warehouse)	Optional FK
warehouse_name	VARCHAR(100)	e.g. 'Dadri Main Store', 'Greater Noida Hub'	Required
address / city / state / pincode	TEXT / VARCHAR	Location details	Required
is_default	BOOLEAN	Default location for this vendor's stock	Default: true for vendor's first warehouse
is_active	BOOLEAN	Location currently operational	Default: true
created_at	DATETIME	Record creation date	Auto-set

4. Inventory Management Flow

Step 1 – Vendor Lists Product + Variants

Vendor creates a Product (category, name, base price, GST), then adds one or more Product Variants (size/color combinations) each with its own SKU and starting stock_qty at their default warehouse_id.

Step 2 – Stock Received / Restocked

When the vendor restocks, a new Stock Ledger row is created with movement_type = 'stock_in', increasing variant.stock_qty and recording balance_after.

Step 3 – Parent Places Order (Reservation)

When a parent checks out, the system creates Stock Ledger rows with movement_type = 'order_reserved', incrementing variant.reserved_qty so the item can't be oversold while payment is processing.

Step 4 – Payment Success → Stock Deducted

On payment confirmation, movement_type = 'order_fulfilled' reduces both stock_qty and reserved_qty for the ordered variants, linked via reference_type = 'order' and reference_id = order_id.

Step 5 – Payment Failure / Cancellation → Release

If payment fails or order is cancelled, reserved_qty is released back (no change to stock_qty) via an 'adjustment' ledger entry.

Step 6 – Returns

On an approved return, a 'return_in' ledger entry adds the quantity back to stock_qty (if the item is resaleable) at the relevant warehouse.

Step 7 – Low Stock Alerts

A scheduled job compares variant.stock_qty against variant.low_stock_threshold and triggers a notification to the vendor (reuses the existing Notification System).

Step 8 – Multi-Warehouse Fulfilment (Future)

If a vendor operates multiple warehouses, the order fulfilment logic picks the warehouse with available stock closest to the delivery address — enabled because Stock Ledger and Variant are already warehouse-aware.

5. Future-Proofing & Scalability Considerations

Generic Category + Attribute system

The Category table (from v2) plus the variant's color/attribute field means new product types (e.g. sports kits, lab coats, art supplies) can be added without altering table structures — only new category rows and variant attributes.

SKU-level inventory

Tracking stock per variant (not per product) supports accurate stock display, prevents overselling specific sizes, and is required for any future barcode/POS or offline-store integration.

Stock Ledger as source of truth

An append-only ledger means stock history is auditable, supports reconciliation, and allows reporting (e.g. 'fastest selling size') without re-architecting later.

Multi-warehouse ready

Even single-location vendors get a default warehouse row now, so scaling to multiple cities, regional hubs, or a platform-owned fulfillment center later is a data change, not a schema change.

Soft status fields everywhere

Using `is_active` / status ENUMs (rather than deleting rows) on Vendor, Product, Variant, and Warehouse preserves historical orders and reports even if items are discontinued.

Partitioning & indexing readiness

High-volume tables (Order, Stock Ledger) should be indexed on (`vendor_id`, `created_at`) and (`school_id`, `created_at`) from day one, and can later be partitioned by `academic_year` or month as order volume grows.

API versioning & caching

REST APIs should be versioned (e.g. `/api/v1/...`) from the start; product catalog reads (heavy, read-mostly) are good candidates for Redis caching, especially the Product Visibility Engine query.

Role-based access control (RBAC)

Admin, Vendor, School, and Parent roles are already separated; adding new roles (e.g. Warehouse Staff, Delivery Partner) later only requires new role entries and permission mappings, not new auth tables.

Event-driven notifications

Keeping notification triggers (low stock, order placed, settlement done) as discrete events makes it easy to plug in new channels later (push notifications, in-app inbox) without touching core inventory/order logic.

6. Updated Total Database Tables (v3)

With Product Variant, Stock Ledger, and Warehouse added to the v2 structure (Admin, Vendor, Category, Product, School, School–Product Mapping & Approval, Parent, Student Profile, Order), the platform now uses 12 core tables.

#	Table Name	Purpose	Key Relationships
1	Admin	Platform admin/sub-admin accounts and roles	Referenced by School.onboarded_by
2	Vendor	Vendor registration, KYC, GST, bank & co	Referenced by Product, Warehouse, Order
3	Category	Master list of product types (uniform, footwear, accessories, etc.)	Referenced by Product
4	Product	Vendor-listed items of ANY category (base info: name, price, GST, description)	vendor_id → Vendor; category_id → Category; referenced by Product Variant
5	Product Variant	NEW – SKU-level size/color variants with their own stock	product_id → Product; referenced by Stock Ledger, Order
6	Warehouse / Location	NEW – vendor or platform stock locations	vendor_id → Vendor; referenced by Stock Ledger
7	Stock Ledger	NEW – full inventory movement history per variant	variant_id → Product Variant; warehouse_id → Warehouse
8	School	School registration, address, classes_offered, status	onboarded_by → Admin; referenced by Student, Mapping & Approval
9	School–Product Mapping & Approval	School admin approval + class/gender mapping for each product	school_id → School; product_id → Product
10	Parent	Guardian login accounts (email/phone OTP)	Referenced by Student, Order
11	Student Profile	Child profile linked to parent + school + class + gender	parent_id → Parent; school_id → School; referenced by Order
12	Order	Order/transaction records with items, payment, GST status	student_id → Student; school_id → School; vendor_id → Vendor; variant_id → Product Variant

Total core tables (v3): 12 — Admin, Vendor, Category, Product, Product Variant (NEW), Warehouse/Location (NEW), Stock Ledger (NEW), School, School–Product Mapping & Approval, Parent, Student Profile, Order.

Note: This is a further-revised design based on the original "School Uniform Marketplace – Platform Design & Technical Specification" (v1.0, June 2026), now incorporating SKU-level inventory management, a stock movement ledger, multi-warehouse support, and general scalability practices for future growth.